

Distributed Cyber-Resilience Platform for Containerized Cloud Applications

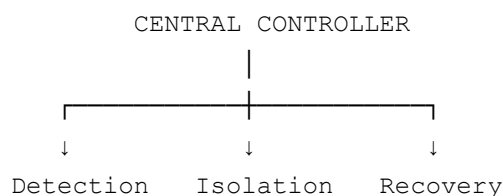
1. One-Paragraph Summary

We are building a cybersecurity platform for cloud/containerized applications that doesn't just *detect* attacks — it completes the full resilience cycle: **Detect → Verify → Isolate → Recover → Validate → Reintegrate**. The key design principle is that the security system itself has **no single point of failure or compromise**. Instead of one central controller, multiple "resilience nodes" monitor each other, independently collect evidence, and collaboratively authorize high-impact actions (like isolating a service). Critically, the platform is explicitly designed to keep working even if **one of its own security nodes is compromised** — a scenario most student/industry cybersecurity projects never test for.

2. The Problem

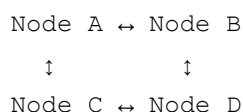
Most cybersecurity systems stop at detection and hand off to a human. Our platform automates the entire loop, and — more importantly — it assumes its **own infrastructure can be attacked too**. A system that protects an application but collapses the moment its own controller is hacked isn't actually resilient; it just moved the weak point.

Centralized architecture risk:



If this controller is hacked, crashed, or overloaded, the *entire* protection mechanism can stop working. That's a single point of failure **and** a single point of compromise.

Our architecture:



No node is a "head." Losing or corrupting one node degrades the system gracefully instead of catastrophically.

3. Fixing the Weaknesses in the Original Design

Before presenting the improved version, here's what was wrong or underspecified in the first draft, and how we've fixed it:

Weakness in v1	Fix in v2
Evidence could be forged — nothing stopped a compromised node from sending fake "evidence" about a healthy node.	Signed, authenticated evidence. Every evidence message is cryptographically signed by the originating node (mTLS + signed payloads). A node's claims can be traced and are non-repudiable, so lying nodes can be caught and penalized.
"Distributed decision/quorum" was really just unlabeled Byzantine Fault Tolerance (BFT) — reinventing it without naming it risks looking naive in a literature review.	We explicitly build on BFT quorum theory (Byzantine agreement tolerates up to $\lfloor (n-1)/3 \rfloor$ faulty nodes) and cite it — our contribution is <i>what feeds into</i> the quorum (typed, weighted evidence), not the quorum mechanism itself.
Detection method (thresholds/statistics) is the least novel part and would attract the most reviewer skepticism.	Keep it simple and <i>say so explicitly</i> in the proposal: detection method is a means to an end, not the contribution. Don't oversell it.
No mechanism for nodes to lose/regain trust over time — a node was either "trusted" or "not," with no continuous scoring.	Added a trust-decay and trust-recovery score per node (see Section 6) — this becomes a core, measurable novelty.
Vague claim that the project is "novel" without grounding in existing literature.	Section 8 below explicitly maps our claims against real prior work (Dempster-Shafer evidence fusion, PBFT proactive recovery, TOLERANCE, distributed IDS trust frameworks) so we're not overclaiming.
No specified domain — attacks/policies were generic.	Locked to one domain (recommend: cloud healthcare or banking app — higher stakes make isolation-aggressiveness tradeoffs meaningful and demoable).
No concrete plan for <i>how</i> the "attack the security system itself"	Section 7 gives a specific experimental protocol for

scenario would actually be run and measured.

this, since it's your headline novelty claim.

4. Improved Architecture

Resilience Node = Application workload + Resilience Agent

Each Resilience Agent has 10 modules:

1. Monitoring Module (CPU, memory, network, syscalls, auth activity, file integrity)
2. Detection Module (threshold/statistical anomaly detection)
3. Threat/Risk Scoring Module
4. Evidence Collector (now: **signed** evidence, typed by category)
5. Peer Communication Module (mTLS-authenticated)
6. Distributed Decision / Quorum Module (BFT-style, $\lfloor (n-1)/3 \rfloor$ fault tolerance)
7. Isolation/Response Module (container quarantine, network policy revocation)
8. Recovery Module (redeploy from known-good image)
9. Validation Module (post-recovery integrity + behavior checks)
10. Reintegration Module (**trust-score-gated**, not instant)

A separate **read-only dashboard** visualizes node status, threat scores, incidents, and trust scores — it never makes decisions itself.

Example deployment (domain: cloud healthcare app):

- Node A: Patient portal (web service)
- Node B: Authentication/identity service
- Node C: Records API service
- Node D: Database service

5. Evidence Model (Improved)

Each evidence message includes:

- Target node
- Observation type (network / process / file-integrity / auth)
- Confidence score (0–1)
- Timestamp
- **Digital signature of the originating node**

Example:

```
Node A → Target: C
Observation: abnormal outbound network traffic
Confidence: 0.88
Signature: [signed with Node A's private key]
```

The quorum module doesn't just count votes — it weighs evidence by **type diversity** (independent evidence types corroborating each other is stronger than the same type repeated) and **confidence**, before checking whether the weighted total crosses the containment threshold. This is more resistant to a small number of colluding or compromised nodes flooding the system with low-quality "evidence."

6. The Trust-Decay / Trust-Recovery Model (Core Novelty #1)

Instead of a binary "trusted / not trusted" state, every node — including resilience nodes themselves — carries a **continuous trust score** (0–100).

- Trust **decreases** when: a node's evidence is repeatedly contradicted by peers, a node is isolated, or a node's claims don't correlate with independently observed behavior.
- Trust **increases gradually** through validated, uneventful behavior over time — not instantly on "recovery complete."
- Reintegration after recovery isn't a single gate — it's a **staged climb**: Quarantine → Restricted access → Monitored operation → Peer-validated → Full access, where each stage requires the trust score to cross a rising threshold, sustained over a time window (not just a one-time check).
- A **compromised resilience node** (not just an app service) is also subject to this — if Node A starts making unsupported accusations, Node A's own trust score drops, and its future evidence is automatically down-weighted by peers (without needing a human to intervene).

This turns "reintegration" from a yes/no gate into a **measurable, quantifiable process** — giving you a real metric (*trust-recovery time, trust trajectory under attack*) that's genuinely uncommon in published resilience systems.

7. Core Novelty #2 — Empirically Attacking the Security Layer Itself

This is your strongest, most defensible claim, so it needs a rigorous protocol, not just a design diagram.

Experimental protocol:

1. Deploy the 4-node platform protecting the chosen domain app in Kubernetes/Minikube.
2. Run baseline attacks (Scenario A — genuine compromise of an app node) and record: time-to-detect, time-to-isolate, recovery time, availability.
3. **Compromise one resilience node itself** (simulate: inject false evidence generation, or actually exploit the agent process) and have it:
 - falsely accuse a healthy node, AND separately
 - refuse to participate / go silent, AND separately
 - attempt to block a legitimate isolation decision.
4. Measure system behavior under each sub-scenario: does the system correctly withhold action (no false isolation), does overall trust-score of the bad node fall, does the app stay available, does detection of the *bad resilience node itself* happen and how fast.
5. Repeat with **2 out of 4 nodes compromised** to find the actual breaking point of your fault tolerance (BFT theory predicts you tolerate up to $\lfloor (n-1)/3 \rfloor$ — with $n=4$ that's 1 node; confirm this experimentally and report it honestly, don't hide the limit).
6. Compare all of the above against the **centralized baseline** (Section 9) under the identical attack.

Reporting the actual breaking point (not just the happy path) is what turns this into a research contribution rather than a demo.

8. Honest Positioning Against Prior Work (put this in your report so you don't overclaim)

Existing work	What it does	How you differ
Dempster-Shafer evidence fusion in IDS (used since 2000s)	Weights alerts from multiple sensors by confidence	You apply this specifically to authorizing containment actions in a container/microservice environment , not just alert scoring
PBFT / Castro-Liskov proactive recovery	Formal BFT consensus + periodic proactive recovery	You use BFT-style quorum as a <i>building block</i> , and add domain-specific, trust-scored reintegration on top
"TOLERANCE" (2025) — automated recovery + replication on intrusion-tolerant consensus	Adds automated recovery to BFT consensus	Your system is tested specifically in a cloud-native containerized domain, with continuous trust scoring rather than binary recovery-complete
Distributed trust frameworks for IDS (2011, Bayesian belief-based)	Distributed trust scoring for detecting compromised hosts	You extend trust scoring to govern reintegration speed , not just detection, and test it under adversarial resilience-node behavior specifically

What you should say in your proposal: "We build on established Byzantine fault-tolerant consensus and evidence-fusion techniques. Our contribution is (1) a continuous trust-decay/recovery model governing progressive reintegration, and (2) an empirical evaluation of the complete detect-to-reintegrate pipeline when the resilience infrastructure itself is the attack target, in a containerized microservice domain." This is defensible and specific — not "nobody has ever done anything like this."

9. Baseline for Comparison

Build a second, minimal version with one central controller doing detection → isolation → recovery. Compromise the controller. Measure the same metrics. This gives you a clean, visual "before vs. after" story for your defense/demo.

10. Metrics to Report

- Time to detect / time to isolate / recovery time

- False isolation rate / false reintegration rate
 - Recovery success rate
 - Availability during attack
 - **Trust-recovery time** (new metric)
 - System behavior when 1 vs 2 of 4 resilience nodes are compromised (fault-tolerance boundary)
-

11. Suggested Tech Stack

- Kubernetes (Minikube/K3s) for the container environment
 - Python for resilience agents
 - gRPC (with mTLS) for signed peer communication
 - Prometheus/Grafana or a custom dashboard for visualization
 - Kubernetes NetworkPolicies for isolation
 - Git for version control
-

12. What NOT to Claim

- Not "the first distributed cybersecurity system" (false)
 - Not "quorum voting is our invention" (it isn't)
 - Not "our system always identifies the correct malicious node" (no system can guarantee this under enough compromised participants)
 - Not "nothing like this exists" — instead, cite the closest prior work and state your specific delta (Section 8)
-

13. One-Line Pitch (for your friend / for a proposal abstract)

"A distributed, trust-aware cyber-resilience platform for containerized cloud applications that survives compromise not only of the applications it protects, but of its own security infrastructure — using signed, weighted evidence corroboration and a continuous trust-

decay/recovery model to govern containment and reintegration decisions, validated experimentally against a centralized baseline under both application-level and security-node-level attacks."